

ShinobuChat 外部数据接入设计

来源文件：学号-姓名-外部数据接入.md

提交文件名：学号-姓名-外部数据接入.md 项目：ShinobuChat 智能桌宠聊天助手
数据源主题：天气数据接入与桌宠行动建议闭环

1. 数据源选择

ShinobuChat 的定位是“陪伴型 AI 桌宠 + 轻量行动助手”。用户在桌面学习、写代码或独处工作时，经常会问“今天适不适合出门”“要不要带伞”“天气会不会影响我的安排”。因此，本项目选择天气 API 作为外部数据源。

- 数据源名称：wttr.in 天气 JSON 接口
- 接入方式：Agent 技能调用只读 curl 请求，或后端 HTTP Client 定时/按需拉取
- 接口示例：<https://wttr.in/HongKong?format=json>
- 关键字段：城市、当前温度、体感温度、天气描述、降雨概率、未来小时预报、更新时间
- 本地持久化：将查询结果摘要保存到会话消息表；建议扩展 external_data_snapshots 表保存结构化快照
- 界面展示：在聊天窗口以 Shinobu 回复展示；异常天气通过 progress/emotion/done SSE 事件主动提醒，并驱动 Live2D 表情

选择天气数据的原因：它与桌面助手的日常场景强相关，数值明确，容易设计异常检测和联动规则，也能自然产生“提醒用户带伞、调整提醒方式、建议改变计划”等行动闭环。

2. 数据流设计

flowchart LR

```
A["用户询问天气<br/>或系统定时拉取"] --> B["weather 技能<br/>调用 wttr.in"]
B --> C["解析 JSON<br/>温度 / 降雨 / 天气描述"]
C --> D["本地持久化<br/>会话消息 + 数据快照"]
D --> E["异常检测<br/>暴雨 / 高温 / 连续变差"]
E --> F["联动规则<br/>回复风格 / 提醒 / 建议调整"]
F --> G["行动输出<br/>聊天通知 + 语音播报 + Live2D 表情"]
```

在当前项目中，天气能力通过 skills/weather/SKILL.md 定义：识别地点后，使用受限 shell_command 调用 curl wttr.in/<city>?format=json，再由 Agent 将 JSON 结果整理成中文建议。查询结论最终通过 MessageService 保存为会话消息，实现基本持久化。

建议补充结构化快照表：

```
class ExternalDataSnapshot(Base):
    __tablename__ = "external_data_snapshots"

    id = mapped_column(UUID(as_uuid=True), primary_key=True, default=uuid4)
    user_id = mapped_column(ForeignKey("users.id"), nullable=False)
```

```

source = mapped_column(String(40), nullable=False) # weather
location = mapped_column(String(120), nullable=False)
payload = mapped_column(JSON, nullable=False)
observed_at = mapped_column(DateTime(timezone=True), nullable=False)
created_at = mapped_column(DateTime(timezone=True), default=local_now)

```

核心接入代码设计：

```

async def fetch_weather(city: str, http: AsyncHttpClient) -> dict:
    safe_city = quote(city.strip())
    url = f"https://wttr.in/{safe_city}?format=json"
    raw = await http.get_bytes(url, timeout=20)
    data = json.loads(raw.decode("utf-8"))
    current = data["current_condition"][0]
    today = data["weather"][0]
    return {
        "city": city,
        "temp_c": int(current["temp_C"]),
        "feels_like_c": int(current["FeelsLikeC"]),
        "condition": current["weatherDesc"][0]["value"],
        "rain_chance": max(int(hour["chanceofrain"]) for hour in today["hourly"]),
        "raw": data,
    }

```

3. L2：异常检测与主动通知规则

规则	条件	主动通知内容	具体建议
降雨异常	rain_chance >= 70	“ 香港未来数小时降雨概率 85%。 ”	建议带伞；如果用户有外出计划，提前 30 分钟提醒。
高温异常	feels_like_c >= 35	“ 当前体感温度 37°C，属于高温不适。 ”	建议减少户外活动、补水；Live2D 切换为关心/提醒表情。
趋势异常	连续 3 次快照中 rain_chance 上升超过 20%	“ 降雨概率连续上升，从 35% 到 78%。 ”	建议调整出行时间，或把提醒模式调为主动提醒。

核心检测代码：

```

def detect_weather_alert(snapshot: dict, history: list[dict]) -> list[dict]:
    alerts = []
    if snapshot["rain_chance"] >= 70:
        alerts.append({
            "type": "rain",
            "level": "warning",
            "message": f"{snapshot['city']} 降雨概率 {snapshot['rain_chance']}%，建议带伞。",
        })
    if snapshot["feels_like_c"] >= 35:
        alerts.append({
            "type": "heat",
            "level": "warning",
            "message": f"体感温度 {snapshot['feels_like_c']}°C，建议减少户外活动并补水。",
        })
    if len(history) >= 2:
        first = history[-2]["rain_chance"]
        last = snapshot["rain_chance"]
        if last - first >= 20:
            alerts.append({
                "type": "rain_trend",
                "level": "notice",
                "message": f"降雨概率从 {first}% 上升到 {last}%，建议调整出行计划。",
            })

```

return alerts

触发后，系统通过聊天面板主动通知用户，并可调用 TTS 播报。若 AI 回复情绪为 [thinking] 或 [surprised]，前端 Live2D 会同步切换表情，形成“数据异常 -> 主动提醒 -> 多模态反馈”的闭环。

4. L3：数据驱动的建议调整规则

联动规则	数据变化	系统自动调整	验证方式
雨天提醒增强	降雨概率 >= 70%	将提醒方式从“低打扰”调整为“主动提醒”；回复中优先给带伞建议	输入雨天测试数据后，状态栏显示雨天提醒，回复包含伞具建议
高温语气调整	体感温度 >= 35°C	AI 回复风格切换为关心型；Live2D emotion 设置为 thinking/surprised	前端收到 emotion 事件并切换 Live2D 表情
好天气鼓励行动	降雨概率 <= 20%，温度 18-28°C	建议散步、通风、短休息；不触发强提醒	正常天气数据下仅给温和建议，不弹出异常通知

核心联动代码：

```
def build_weather_action(snapshot: dict, alerts: list[dict]) -> dict:
    if any(item["type"] == "rain" for item in alerts):
        return {
            "emotion": "thinking",
            "reminder_mode": "active",
            "reply": f'{snapshot["city"]} 降雨概率 {snapshot["rain_chance"]}%，出门前记得带伞。',
        }
    if any(item["type"] == "heat" for item in alerts):
        return {
            "emotion": "surprised",
            "reminder_mode": "care",
            "reply": f'体感 {snapshot["feels_like_c"]}°C，今天尽量少晒太阳，多喝水。',
        }
    return {
        "emotion": "happy",
        "reminder_mode": "quiet",
        "reply": f'天气较稳定，{snapshot["temp_c"]}°C，适合安排短时间外出。',
    }
```

5. 至少 5 条测试数据

编号	城市	温度	体感	降雨概率	类型	预期结果
T1	HongKong	26	28	15%	正常数据	不触发异常；建议适合短时间外出
T2	HongKong	28	31	85%	异常数据	触发降雨异常；主动提醒带伞
T3	Guangzhou	34	39	20%	异常数据	触发高温异常；建议减少户外活动
T4	Shenzhen	27	29	70%	边界数据	刚好触发降雨阈值；提醒带伞
T5	Tokyo	18	17	0%	边界数据	不触发异常；回复温和建议

编号	城市	温度	体感	降雨概率	类型	预期结果
T6	HongKong	25	26	35%→58%→78%	趋势数据	触发连续上升趋势提醒

6. 展示效果说明

作业演示时可展示以下界面：

- 1 在 ShinobuChat 聊天框输入：“查一下明天香港会不会下雨。”
- 2 右侧聊天面板显示天气摘要：温度、体感温度、降雨概率、具体建议。
- 3 当测试数据中降雨概率达到 85% 时，系统主动回复：“香港未来数小时降雨概率 85%，建议带伞。”
- 4 前端状态栏显示 Agent/技能执行进度。
- 5 Live2D 根据 emotion 事件切换到提醒或思考表情；开启语音时同时播报提醒。

截图建议：

- 截图 1：天气查询后的聊天回复。
- 截图 2：异常降雨触发主动通知。
- 截图 3：Live2D 表情或状态栏变化。

7. 加分项对应

加分项	项目体现
异常检测 + 建议调整	降雨/高温检测后自动调整提醒模式和回复建议
趋势分析	连续天气快照检测降雨概率上升
四级降级数据获取	优先实时 API；失败后用最近本地快照；再用会话摘要；最后提示用户数据不可用
自动日报/周报	可结合 conversation_digest 和 evening_review 技能，把一周天气影响和用户行动建议整理成复盘

四级降级设计：

实时 wttr.in API
-> 最近一次 external_data_snapshots 快照
-> 最近会话中的天气摘要
-> 明确提示“暂时无法获取天气数据”，只给通用建议

8. 总结

本设计围绕 ShinobuChat 的桌宠助手定位，选择天气数据作为外部数据源，实现了从“数据接入、结构化解析、本地持久化、界面展示”到“异常检测、主动通知、建议调整、Live2D/语音反馈”的完整闭环。

它的重点不是简单调用天气 API，而是让外部数据真正影响助手行为：下雨时主动提醒带伞，高温时调整为关心语气，天气稳定时鼓励用户安排短暂外出。这样，ShinobuChat

从一个会聊天的桌宠，进一步变成了能感知外部环境并主动帮助用户调整计划的智能应用。